Язык программирования Python

Функции

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Пользовательские функции

Вполне можно написать программу, используя исключительно встроенные функции, типы данных и управляющие структуры (циклы и условия). Однако очень часто возникает необходимость **повторно выполнять одну и ту же обработку**, но в разных местах программы и с некоторыми **отличиями в начальных значениях**.

Python позволяет оформить такие участки кода в виде отдельных пользовательских функций, которым передаются начальные значения (**аргументы**) для вычислений.

Более формальное определение функции:

Функция – это именованный фрагмент программы, принимающий данные, обрабатывающий эти данные определённым образом и возвращающий значение – результат обработки.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис объявления функции

Для создания функции используется ключевое слово **def**, после которого указывается **имя функции и список параметров в круглых скобках** (переменные, в которые будут записаны данные для обработки).

Вся первая строка называется **заголовком функции**. Далее с отступом в четыре пробела следует блок инструкций (**тело функции**), обрабатывающий параметры:

```
def имя_функции ( парметр_1, параметр_2, ... ) :
    инструкции,_обрабатывающие_параметры
```

Если из функции нужно вернуть результат работы, то используется **оператор возврата return**, после которого перечисляются возвращаемые значения:

```
def имя_функции ( парметр_1, параметр_2, ... ) :
    инструкции,_обрабатывающие_параметры
    return возвращаемое_значение
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Вызов (использование) функции

Функция представляет собой отдельный объект, который можно вызвать (обратиться) из другого места программы.

При вызове функции (т.е. её использовании) нужно знать какие данные она принимает (**аргументы** – переменные с конкретными значениями) и что возвращает в качестве результата. Как из данных получается результат **при вызове** знать не нужно.

Если функция ничего не возвращает, то её можно вызвать так:

```
имя_функции(аргумент_1, аргумент_2, ...)
```

Если функция возвращает результат, то его можно присвоить какой-нибудь переменной:

```
переменная = имя_функции(аргумент_1, аргумент_2, ...)
```

Обратите внимание! Функция в большинстве случаев именно возвращает значение, а не тут же выводит его на экран.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Параметры и аргументы (1)

При работе с функциями нужно различать понятия «**параметр**» и «**аргумент**».

```
def add(x, y):  # Объявление функции
    return x + y
r = add(3, 4)   # Вызов функции
```

При объявлении функции вы указываете её **параметры** (**x и y**). Параметр – это объект, который существует исключительно внутри функции. Обратиться к нему из основной программы или из других функций нельзя. Он уничтожается сразу после исполнения функции.

При вызове функции **числа 3 и 4** – это **аргументы** (фактические параметры) функции. Ими могут быть любые выражения, определенные ранее переменные, константы, другие функции и т.д. Функция при вызове записывает число 3 в свой параметр x, а число 4 – в параметр y.

Имена параметров никак не связаны с именами аргументов!

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Параметры и аргументы (2)

При работе с функцией также следует помнить, что она будет работать для всех объектов, которые поддерживают использованные операции.

Например, функцию сложения можно использовать как для чисел, так и для строк, и вообще любого другого типа, поддерживающего операцию «+»:

```
>>> def add(x, y):
        return x + y

>>> add(3, 4)
7
>>> add(1.4, 4.1)
5.5
>>> add('а', 'б')
'аб'
```

Произошло объединение строк в одну

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример пользовательской функции (1)

Как правило, все пользовательские функции объявляются в начале файла с программой и не перемешиваются с основным кодом этой программы.

Функция, переводящая градусы по Фаренгейту в градусы по Цельсию:

Осмысленное имя (выбирает программист)

Параметры (через запятую, если несколько)

```
def convert_to_cels(fahren):
    return (fahren - 32) * 5/9
```

Возвращаемое значение (после return никакой код в блоке не работает)

Вызов этой функции должен быть после её объявления.

Выглядеть он может так:

```
cels = convert_to_cels(80)
print('Ответ:', cels)
# Ответ: 26.666666666666668
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример пользовательской функции (2)

Функции можно вызывать, передавая их друг другу в качестве аргументов. Например:

```
print(convert_to_cels(float(input('Градусы в F: '))))
# Градусы в F: 71.6
# 21.999999999999996
```

1. Сначала вызывается функция input(), которая возвращает строку с числом. Например, **'71.6'**
2. Затем функция float() преобразует эту строку в вещественное число: **71.6**
3. После этого пользовательская функция convert_to_cels() переводит это число в градусы Цельсия: **21.999999999999996**
4. Функция print() выводит полученное значение на экран.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример двух реализаций одной функции

Необходимо подсчитать сумму цифр целого числа. Это можно сделать разными способами:

```
def sum_digits(n):
    sumd = 0
    while n != 0:
        sumd += n % 10
        n = n // 10
    return sumd
```

```
def sum_digits(n):
    str_num = str(n)
    sumd = 0
    for i in str_num:
        sumd += int(i)
    return sumd
```

Преобразов. к указанному типу

```
# основная программа одинакова для обоих случаев
dg = int(input('Введите число: '))
sm = sum_digits(dg)
print('Сумма цифр = ', sm)
```

```
Ответ:
Введите число: 12345
Сумма цифр = 15
```

Несмотря на разную реализацию, вызов функций одинаков, результат выполнения тоже. Отличие – время выполнения (второй вариант на 35% дольше из-за преобразований число-строка-число).

Таким образом, один из плюсов функций – **возможность изменить реализацию без изменения существующего кода**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Значения параметров по умолчанию

В общем случае количество параметров и аргументов должно совпадать. Однако при объявлении функции параметру можно присвоить значение, которое будет использоваться при вызове функции без передачи значения, соответствующего этому параметру.

Например, необходимо написать функцию, которая будет возводить число **x** в заданную целую степень **y**. Если степень не указана, то необходимо возводить в квадрат.

```
def power(x, y=2):
    res = 1
    for i in range(y):
        res *= x
    return res
a = 3
print(a, ' в квадрате = ', power(a))
b, c = 2, 3
print(b, ' в степени ', c, ' = ', power(b, c))
```

Параметр по умолчанию должны **передаваться самыми последними** (после всех обязательных значений).

```
Ответ:
3 в квадрате = 9
2 в степени 3 = 8
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Безымянная функция (lambda-функция)

Помимо обычных, в Python можно создавать lambda-функции или безымянные функции. Это функции без имени, состоящие из одного выражения, результат которого возвращается автоматически.

Синтаксис объявления следующий:

```
lambda параметры: выражение с обработкой параметров
```

lambda-функцию можно присвоить обычной переменной, после чего использовать её как имя функции:

```
>>> func = lambda x: x**2
>>> func(3)
9
```

Здесь **x** – это параметр; **x**2** – выражение, значение которого будет возвращено.

Чаще всего такие функции используются, когда нужно в качестве аргумента передать функцию. В этом случае они объявляются сразу в строке передачи аргументов.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Строки документации. Функция help()

Для каждой встроенной функции можно получить справку с описанием назначения функции, её параметров и возвращаемого значения. Для этого используется функция **help()**, которой в качестве параметра передаётся имя интересующей функции:

```
>>> help(abs)
Help on built-in function abs in module builtins:
abs(x, /)
    Return the absolute value of the argument.
```

Для того, чтобы подобное описание появлялось и к вашей функции, при её объявлении необходимо добавить строку с описанием:

```
def имя_функции ( парметр_1, параметр_2, ... ) :
    """Описание функции"""
    инструкции_функции
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Параметры функции print() (1)

Мы часто использовали функцию print(), передавая ей одно или несколько значений для отображения на экране. При этом несколько значений выводились на экран через пробел в одну строку.

При повторном вызове print() выводит значения на новой строке:

```
print(1, 3, 2)
print('4', 0, '7')
```

```
Ответ:
1 3 2
4 0 7
```

На самом деле у функции print() есть несколько «скрытых» параметров, которым заданы значения по умолчанию. Чтобы их посмотреть можно воспользоваться справкой:

```
>>> help (print)
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

Перечисляем объекты, которые хотим отобразить.

Строка-разделитель между объектами. По умолчанию пробел.

Строка, которая располагается после последнего объекта. По умолчанию - перевод на новую строку.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Параметры функции print() (2)

Таким образом, мы можем сами задать строку-разделитель, а также строку-окончание.

Для этого при вызове функции print() нужно явным образом прописать название параметра и присвоить ему новое значение:

```
print(1, 3, 2, sep=':')
```

```
Ответ:
1:3:2
```

```
print('4', 0, '7', sep=',', end='_')
print('строка 1', 'число 2', sep=',')
```

```
Ответ:
4,0,7_строка 1,число 2
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Модули

Предположим, что вы написали несколько полезных функций, которые часто используете в своих программах. Копировать их из прошлой программы и вставлять в новые достаточно неудобно и долго. Гораздо проще поместить эти функции в отдельный файл, к которому будут иметь доступ все ваши программы, и загружать их оттуда.

Такой файл с кодом называют **модулем**. Модули предназначены для того, чтобы в них хранить часто используемые функции, константы, классы и импортировать (подключать) их в программы.

Можно условно разделить код на модули и программы:

- **программы** предназначены для непосредственного запуска;
- **модули –** для импортирования их в другие программы.

Стоит заметить, что модули могут быть написаны не только на языке Python , но и на других языках (например, на C++).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Импортирование модуля

Чтобы воспользоваться функциями и другими объектами, которые находятся в модуле, его необходимо импортировать с помощью инструкции **import**.

Например, подключим модуль **math** с математическими функциями:

```
>>> import math   # пишем import и имя модуля
```

После этой команды в память загружается стандартный модуль math. Теперь можно обращаться к функциям, находящимся в этом модуле. Для этого мы указываем имя модуля и, через точку, имя функции с аргументами:

```
>>> math.sqrt(9)   # корень квадратный из 9
3.0
```

Узнать о функциях, которые содержатся в модуле, можно с помощью функции help():

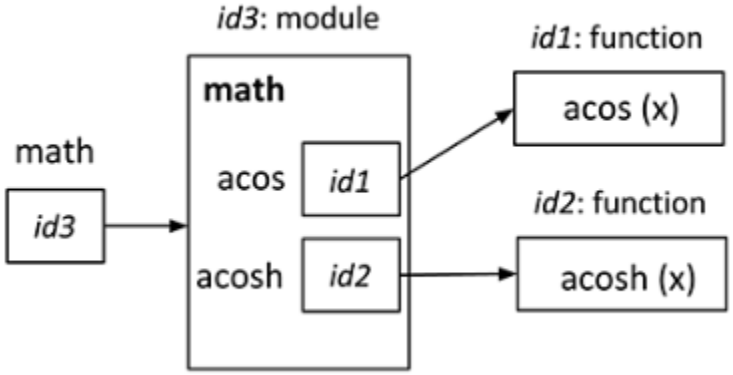
```
>>> help(math)
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Принцип импортирования

В момент импортирования модуля math создаётся переменная с таким же именем **math**:

```
>>> import math
>>> type(math)
<class 'module'>
```



Функция type() показала, что у переменной math тип данных – модуль (module). Переменная math содержит ссылку на модульный объект, внутри которого есть ссылки на функции.

В момент вызова функции sqrt() Python находит переменную math (модуль должен быть предварительно импортирован), просматривает модульный объект, находит функцию sqrt() внутри этого модуля и затем вызывает её.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Функции модуля math (1)

В библиотеке **math** содержится достаточно большое количество часто используемых математических функций, которые не нужно реализовать самостоятельно.

Функция	Назначение функции
`math.ceil(x)`	Возвращает ближайшее целое число большее, чем x. >>> math.ceil(3.2) # Ответ: 4
`math.floor(x)`	Возвращает ближайшее целое число меньшее, чем x. >>> math.floor(3.2) # Ответ: 3
`math.fabs(x)`	Возвращает абсолютное значение числа. >>> math.fabs(- 7) # Ответ: 7.0
`math.factorial(x)`	Вычисляет факториал x. >>> math.factorial(5) # Ответ: 120
`math.exp(x)`	Вычисляет e**x. >>> math.exp(3) # Ответ: 20.08553692318766
`math.sqrt(x)`	Корень квадратный от x. >>> math.sqrt(25) # Ответ: 5.0
`math.pow(x, y)`	Вычисляет x**y. >>> math.pow(2, 10) # Ответ: 1024.0

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Функции модуля math (2)

Функция	Назначение функции
`math.log2(x)`	Логарифм по основанию 2. >>> math.log2(8) # Ответ: 3.0
`math.log10(x)`	Логарифм по основанию 10. >>> math.log10(1000) # Ответ: 3.0
`math.sin(x)`	Синус x
`math.cos(x)`	Косинус x
`math.tan(x)`	Тангенс x

Необходимо учитывать, что тригонометрические функции принимают угол в радианах. Для преобразования углов используются две функции:
`math.degrees(x)` – преобразует x из радиан в градусы.
`math.radians(x)` – преобразует x из градусов в радианы.

Также в модуле math содержится несколько констант:
`math.pi` – число пи.
`math.e` – число e.

Другие функции модуля можно увидеть в справке или на официальном сайте (https://docs.python.org/3/library/math.html).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Импортирование отдельной функции

В Python можно импортировать из модуля отдельную функцию:

```
>>> from math import sqrt
```

* – импортирование из модуля всех имён.

При таком импортировании переменная math не будет создана. В память загрузится только функция sqrt(), что позволяет при её использовании не вводить имя модуля:

```
>>> sqrt(25)   # имя модуля не указываем
5.0
```

Но нужно быть внимательным, чтобы случайно не переопределить названия уже имеющихся функций:

```
>>> def sqrt(x):
      return x*x

>>> sqrt(6)  # здесь вызывается наша функция
36
>>> from math import sqrt
>>> sqrt(9)  # а здесь уже импортированная
3.0
```

Происходит подмена значения переменной sqrt.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Импортирование через псевдоним

При импортировании модуля ему можно присвоить **новое имя (псевдоним)**, которое будет использоваться в программе. Чаще всего это делают для модулей с длинными названиями.

Например, модуль **random**, содержащий функции для генерации псевдослучайных значений:

```
>>> import random as rnd
```

При таком импортировании в памяти будет создана переменная **rnd**, а не random. Обращение к функциям происходит через новое имя rnd.

Например, функция **randint()** из модуля random, возвращающая случайное целое число из указанного диапазона, будет вызываться так:

```
>>> rnd.randint(0, 100)
54
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Встроенные функции. Модуль __builtins__

Ранее в программах мы уже использовали такие функции, как input(), print(), int(), float() и т.д.. На самом деле эти функции также находятся в модуле, который Python загружает в память в момент начала своей работы. Этот модуль называется **__builtins__** (по два нижних подчёркивания в начале и конце имени модуля).

Чтобы узнать описание функций из этого модуля можно использовать функцию help():

```
>>> help(__builtins__)
```

Чтобы получить полный перечень имён функций и переменных, содержащихся в модуле, можно использовать функцию dir():

```
>>> dir(__builtins__)
[..., 'range', 'repr', 'reversed', 'round', 'set',
'setattr', 'slice', 'sorted', 'staticmethod', 'str',
'sum', 'super', 'tuple', 'type', 'vars', 'zip']
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Создание собственного модуля

Для создания своего модуля вам достаточно вынести в отдельный текстовый файл с расширением **.py** функции и другие необходимые объекты (желательно, логически связанные друг с другом).

Например, создадим файл с именем **mm.py**, содержащий следующий исходный код:

```
def f():
    return 42
```

Для использования этого модуля в своей программе его нужно импортировать (**название модуля совпадает с названием файла**). При этом есть два варианта размещения файла:

1. Сохранить файл mm.py в той же папке, где будет располагаться ваша программа. При создании новой программы нужно копировать файл модуля в новую папку.
2. Сохранить файл mm.py в одну из папок, где Python хранит свои модули. В этом случае все ваши программы на компьютере будут иметь доступ к этому модулю без дополнительного копирования.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Размещение собственного модуля

Для выяснения того, где Python хранит свои модули, нам понадобится модуль **sys** и переменная **path** внутри него:

```
>>> import sys
>>> sys.path
['', 'C:\\Python38\\Lib\\idlelib',
'C:\\Python38\\python38.zip', 'C:\\Python38\\DLLs',
'C:\\Python38\\lib', 'C:\\Python38',
'C:\\Python38\\lib\\site-packages']
```

Список папок может отличаться.

Поместим наш файл mm.py в одну из этих папок. Например, в 'C:\\Python38'. Если всё сделано правильно, то можно из любой программы импортировать наш модуль следующим образом:

```
>>> import mm   # указываем имя без расширения
>>> mm.f()   # вызываем функцию f() из модуля
42
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Тестирование модуля и импортирование

Проведём эксперимент. Создадим модуль **mypr.py** с объявлением функции и кодом для её тестирования:

```
def func(x):
    return x**2 + 2

x = float(input('Введите значение: '))
print(func(x))
```

Теперь импортируем модуль в интерактивном режиме:

```
>>> import mypr
Введите значение: 10
102.0
```

Импортирование модуля привело к выполнению инструкций внутри этого модуля. Однако в большинстве случаев этого делать не нужно.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Выполнение кода при импортировании

Чтобы отделить исполнение модуля (**Run Module**) от его импортирования (**import mypr**), существует специальная переменная **__name__**.

При выполнении модуля переменная **__name__** будет содержать строку "**__main__**", а в случае импортирования – строку с именем модуля ("**mypr**").

Перепишем модуль mypr.py с учётом этой особенности:

```
def func(x):
    return x**2 + 2

if __name__ == '__main__':
    x = float(input('Введите значение: '))
    print(func(x))
```

Также можно создать функцию main() с кодом для тестирования.

```
if __name__ == '__main__':
    main()
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Стандартные модули

Интерпретатор Python скачивается вместе с набором стандартных модулей. Полное описание вы можете посмотреть в документации: https://docs.python.org/3/library/

Некоторые из них:

- math – стандартные математические функции.
- random – генерация псевдослучайных значений.
- statistics – функции математической статистики (среднее, дисперсия, корреляция и т.д.).
- datetime – содержит типы для работы с датами и временем.
- copy – создание полных копий объектов коллекций.
- string – дополнительный функционал для работы со строками.
- csv – работа с csv-файлами (можно преобразовать в Excel).
- os.path – работа с путями (манипуляции с папками и файлами).
- pickle – чтение и запись данных из программы в бинарные файлы.
- tkinter – создание оконного приложения.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева